Zakarya
MANAR

BUT1 INFO
Euros

Rapport de la SAE S1.04

Création d'une base de données : Freedom in the World

Année 2023-2024

Sommaire

Sommaire . . . . . *Page 2*

Script manuel de la création de la base de donnée . . . . . *Page 3*

Modélisation de script et création « avec AGL » . . . . . *Page 4*

Peuplement des tables . . . . . *Page 9*

I°/ Script manuel de la création de la base de donnée

Voici le script de création de la table region :

Clé primaire : region_code
Clé(s) étrangères : ---

```
CREATE TABLE region (
     region_code INTEGER PRIMARY KEY,
     name VARCHAR NOT NULL
) ;
```

Voici le script de création de la table status :

Clé primaire : status
Clé(s) étrangères : ---

```
CREATE TABLE status (
     status VARCHAR PRIMARY KEY
) ;
```

Voici le script de création de la table country :

Clé primaire : id_country
Clé(s) étrangères : region_code de la table region

```
CREATE TABLE country (
     id_country SERIAL PRIMARY KEY,
     name VARCHAR NOT NULL,
     region_code INTEGER REFERENCES region,
     is_ldc INTEGER NOT NULL
) ;
```

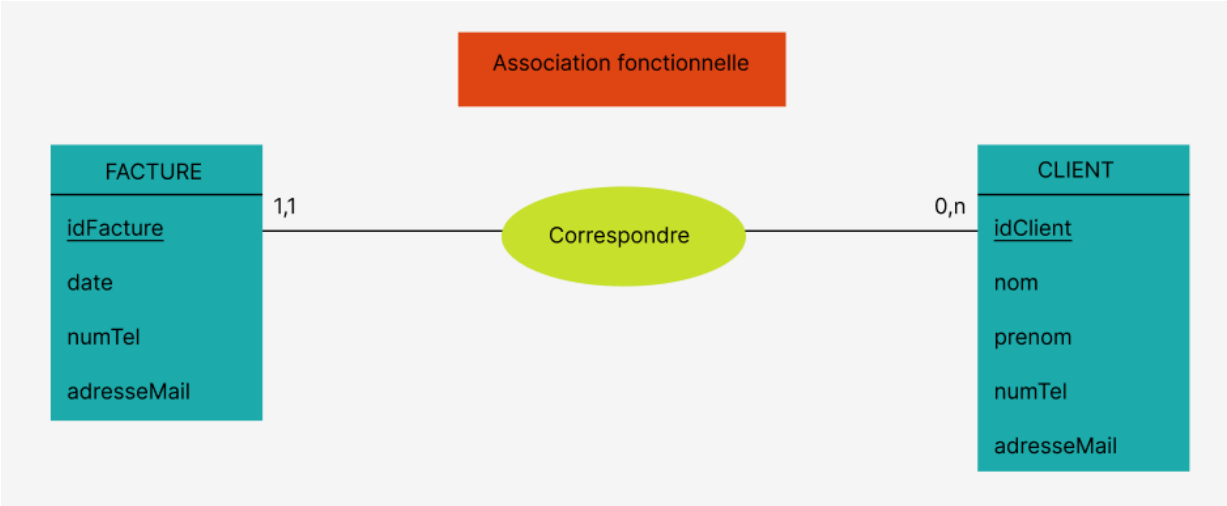
Voici le script de création de la table freedom :

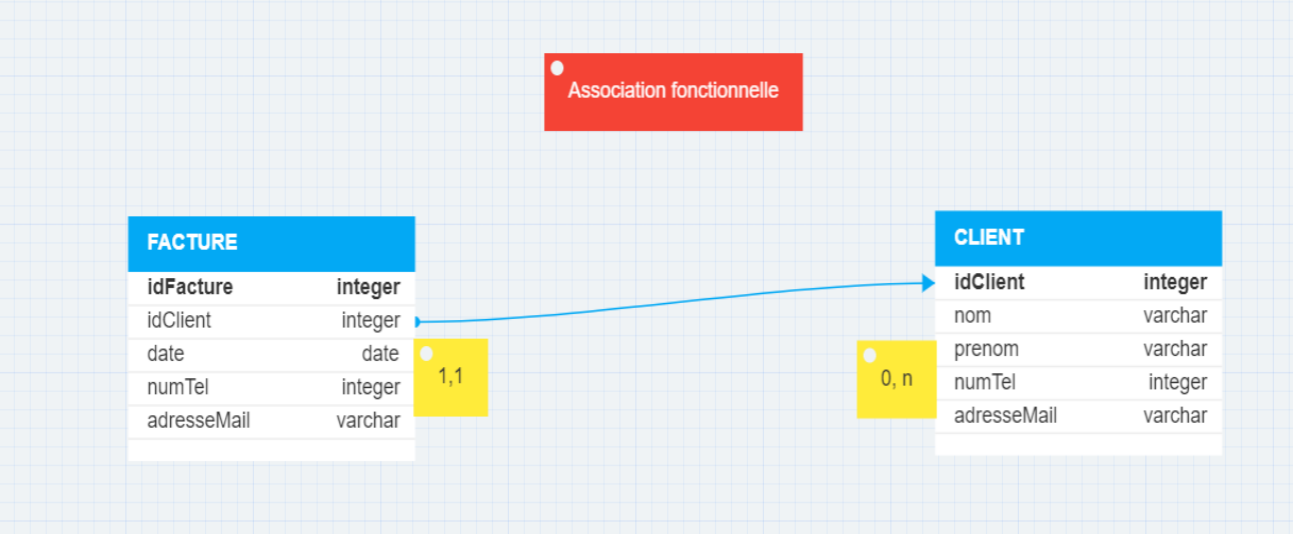
Clé primaire : id_country, year
Clé(s) étrangères : id_country de la table country, status de la table status

```
CREATE TABLE freedom (
     id_country INTEGER REFERENCES country,
     year INTEGER NOT NULL,
     civil_liberties INTEGER NOT NULL,
     political_rights INTEGER NOT NULL,
     status VARCHAR REFERENCES status,
     PRIMARY KEY (id_country, year)
) ;
```

II°/ Modélisation et script de création " avec AGL "

1) Illustrations comparatives cours/AGL commentée d'un association fonctionnelle



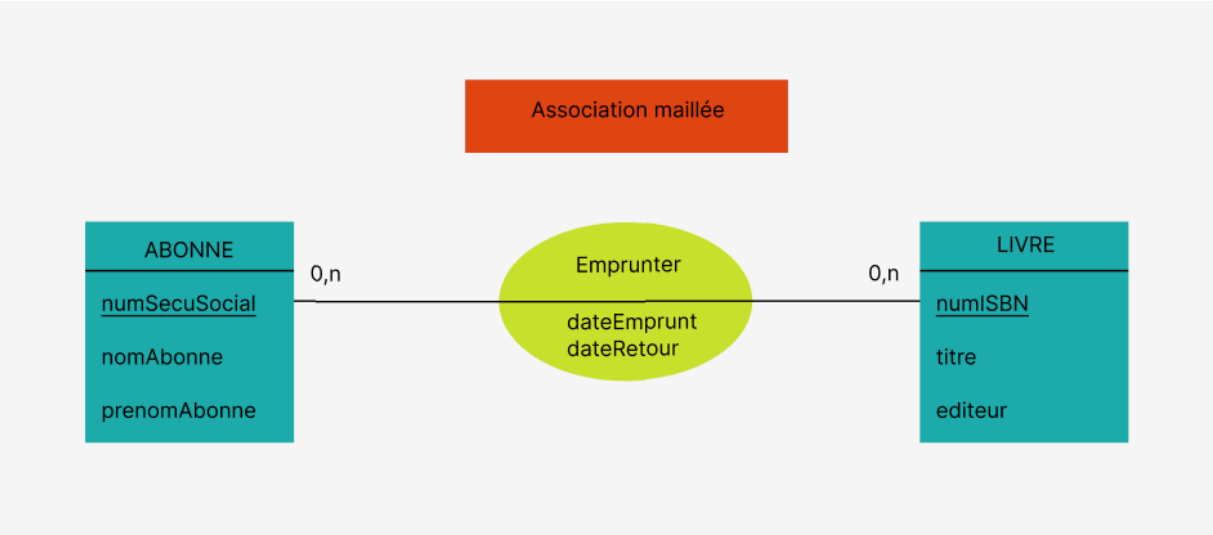


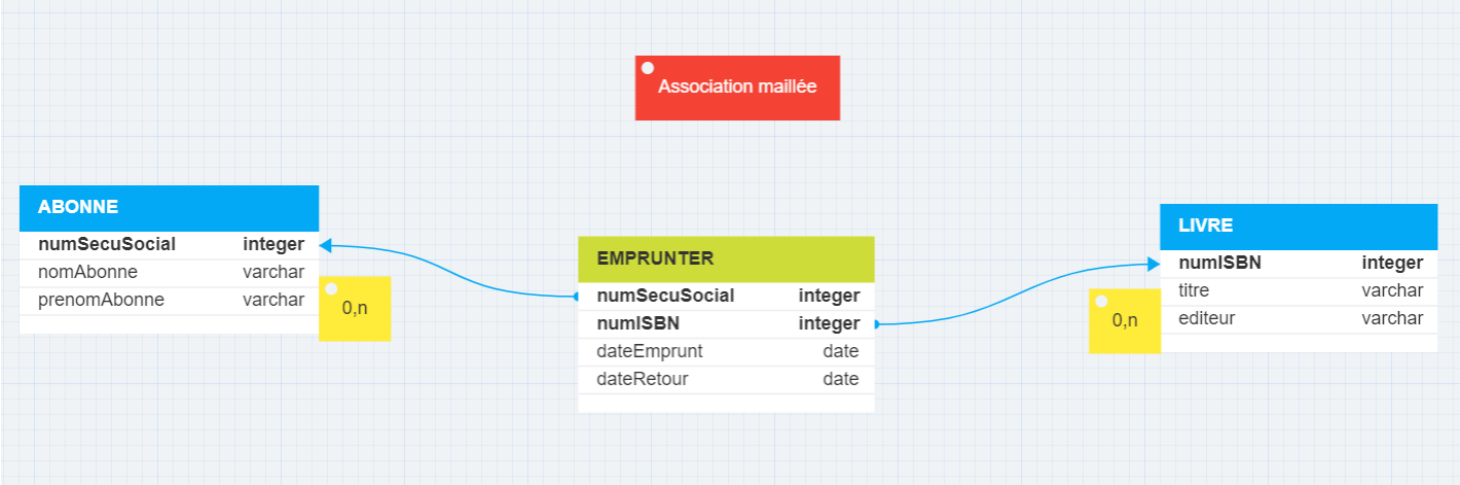
Dans ce modèle entité-association, la table FACTURE rassemble les détails particuliers de chaque facture, tels que la date d'émission, le numéro de téléphone et l'adresse e-mail associés. Chaque facture est directement liée à un unique client, représentant une relation un-à-un.
D'autre part, la table CLIENT conserve des informations générales sur les clients, incluant leur nom, prénom, numéro de téléphone et adresse e-mail. L'association "Correspondre" est établie pour refléter la relation entre les factures et les clients. Chaque enregistrement dans "Correspondre" relie une facture à son client associé, assurant ainsi une correspondance précise. La cardinalité est définie comme un-à-un de FACTURE vers "Correspondre" (1,1) et zéro-à-plusieurs de CLIENT vers "Correspondre" (0,n). Ainsi, chaque facture a une correspondance unique dans "Correspondre", tandis qu'un client peut être associé à zéro ou plusieurs factures.

En résumé, ce modèle permet de représenter la relation entre les factures et les clients de manière précise, en spécifiant la liaison exclusive entre chaque facture et son client, tout en permettant à un client d'être associé à plusieurs factures.

La première image est un modèle type entité-association fonctionnel représenté par les deux tables et l’association au milieu. Pour la seconde image, il s’agit d’un schéma relationnel représenté par les deux tables où l’on a ajouté dans la table FACTURE l’attribut idClient qui est une clé étrangère et qui pointe vers la clé primaire idClient de la table CLIENT.

2) Illustrations comparatives cours/AGL commentée d'un association maillée





Dans ce modèle d'association maillée, la table ABONNE recueille des informations sur les abonnés, comprenant leur numéro de sécurité sociale, nom et prénom. En parallèle, la table LIVRE stocke des détails sur les livres tels que le numéro ISBN, le titre et l'éditeur.

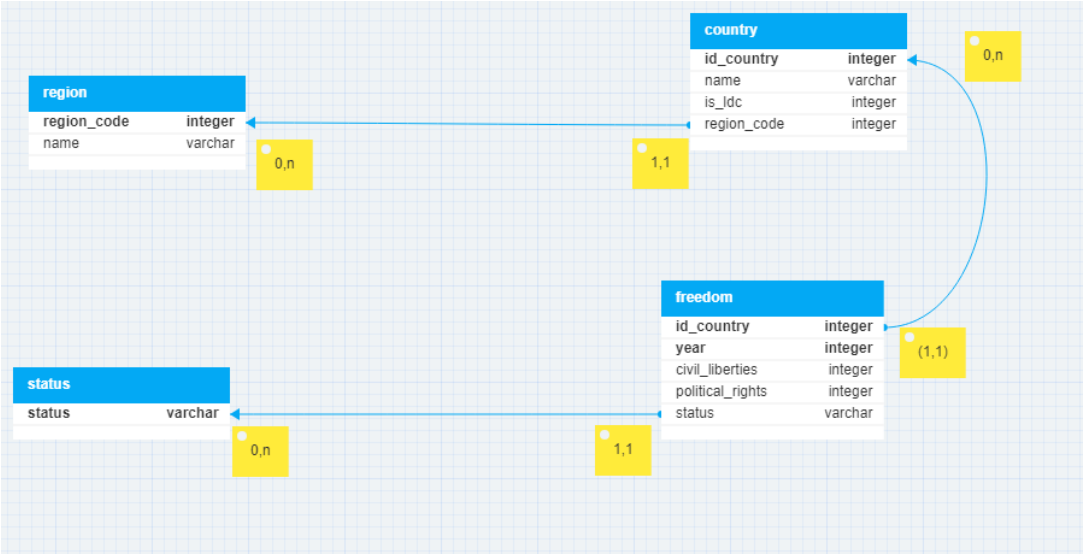
L'association "Emprunter" est établie pour représenter les emprunts de livres par les abonnés. Chaque enregistrement dans cette association est caractérisé par des attributs tels que la date d'emprunt et la date de retour. La cardinalité est définie comme zéro-à-plusieurs (0,n) pour les deux tables (ABONNE et LIVRE). Cela signifie qu'un abonné peut emprunter zéro ou plusieurs livres, et qu'un livre peut être emprunté par zéro ou plusieurs abonnés.

En résumé, ce modèle d'association maillée permet de représenter les relations entre les abonnés, les livres et les emprunts. Il offre la possibilité qu'un abonné puisse emprunter plusieurs livres et qu'un livre puisse être emprunté par plusieurs abonnés.

La première image est un modèle type entité-association maillé représenté par les deux tables et l'association au milieu. Pour la seconde image, il s'agit d'un schéma relationnel représenté par les deux tables et la table EMPRUNTER qui fait l'association entre les deux autres tables à laquelle on ajoute les attribut numSecuSocial et numISBN (qui sont donc le couple de clé primaire de la table EMPRUNTER) qui sont des clés étrangères et qui pointent vers la clé primaire des tables ABONNE et LIVRE.

Pour les deux types d'associations (fonctionnelle et maillée), le logiciel d'AGL utilisé est DB Designer et le logiciel de design manuel est Figma.

3) Modèle physique de données réalisé avec l'AGL



4) Script SQL de création des tables généré automatiquement par l'AGL

Voici le script qui a été généré par DB Designer à la suite du modèle physique des données ci-dessus :

```
CREATE TABLE region (
     region_code INT NOT NULL,
     name varchar NOT NULL,
     PRIMARY KEY (region_code)
) ;
```

```sql
CREATE TABLE country (
     id_country INT NOT NULL,
     name varchar NOT NULL,
     is_ldc INT NOT NULL,
     region_code INT NOT NULL,
     PRIMARY KEY (id_country)
) ;

CREATE TABLE status (
     status varchar NOT NULL,
     PRIMARY KEY (status)
) ;

CREATE TABLE freedom (
     id_country INT NOT NULL,
     year INT NOT NULL,
     civil_liberties INT NOT NULL,
     political_rights INT NOT NULL,
     status varchar NOT NULL,
     PRIMARY KEY (id_country)
) ;

ALTER TABLE country ADD CONSTRAINT country_fk0 FOREIGN KEY
(region_code) REFERENCES region(region_code) ;

ALTER TABLE freedom ADD CONSTRAINT freedom_fk0 FOREIGN KEY
(id_country) REFERENCES country(id_country) ;

ALTER TABLE freedom ADD CONSTRAINT freedom_fk1 FOREIGN KEY
(status) REFERENCES status(status) ;
```

5) Discussion sur les différences entre les scripts produits manuellement et automatiquement

On remarque deux différences. La première réside dans l'affectation de la clé primaire des tables. En effet, on remarque que dans le script produit manuellement, l'affectation de la clé primaire se fait en même temps que l'attribut. Par exemple dans la table status, on crée l'attribut status de cette manière : `status VARCHAR PRIMARY KEY`
En créant l'attribut de cette manière, il sera considéré comme clé primaire dès sa création.
Cependant, dans le script généré automatiquement, on peut voir que l'attribut status est crée dans un premier temps, et dans un second temps il est affecté comme clé primaire. Il est crée de cette manière :

```sql
status varchar NOT NULL
```

puis affecté en clé primaire de cette manière :

```sql
PRIMARY KEY (status)
```

Il est donc crée et affecté comme clé primaire en deux temps alors que dans le script produit manuellement, status est crée et affecté comme clé primaire en une seule fois.

En ce qui concerne la deuxième différence, elle réside dans l'affectation des clé étrangères. En effet, on remarque que dans le script produit manuellement, par exemple pour la clé étangère status qui se situe dans la table freedom, l'attribut status est directement affecté comme clé étrangère : `status VARCHAR REFERENCES status`
Or, dans le script généré par l'AGL, l'attribut status est crée et affecté comme clé étrangère en deux temps. Il est crée dans la table freedom avec `status varchar NOT NULL` puis ilest affecté comme clé étrangère avec :

```
ALTER TABLE freedom ADD CONSTRAINT freedom_fk1 FOREIGN KEY
(status) REFERENCES status(status) ;
```

Il est donc crée et affecté comme clé étrangère en deux temps alors que dans le script produit manuellement, status est crée et affecté comme clé étrangère en une seule fois.

III°/ Peuplement des tables

```
CREATE TABLE temporary (
     country VARCHAR NOT NULL,
     year INTEGER NOT NULL,
     civil_liberties INTEGER NOT NULL,
     political_rights INTEGER NOT NULL,
     status VARCHAR NOT NULL,
     region_code INTEGER NOT NULL,
     region_name VARCHAR NOT NULL,
     is_ldc INTEGER NOT NULL
) ;

\copy temporary FROM chemin/freedom.csv WITH CSV HEADER DELIMITER
' , ' ;

INSERT INTO region SELECT DISTINCT region_code, region_name FROM
temporary ;

INSERT INTO status SELECT DISTINCT status FROM temporary ;

INSERT INTO country (name, region_code, is_ldc) SELECT DISTINCT
country, region_code, is_ldc FROM temporary ;
```

Pour commencer, on crée une table temporary qui stockera les données du fichier freedom.csv afin que l'on puisse insérer ces données par la suite dans les tables region, status et country.

La première ligne de commande SQL copie les données du fichier CSV "chemin/freedom.csv" dans une table temporaire appelée temporary. Le fichier CSV est situé à l'emplacement spécifié par "chemin/". La première ligne du fichier CSV contient les noms de colonnes, et les données sont séparées par des virgules.

La deuxième ligne de commande SQL insère des données (sans doublons grâce à l'instruction DISTINCT) de la table temporaire dans la table region. Elle sélectionne les colonnes region_code et region_name de la table temporary et les insère dans la table region.

La troisième ligne decommande SQL insère les valeurs (sans doublons grâce à l'instruction DISTINCT) de la colonne status de la table temporary dans la table status.

La dernière ligne de commande SQL insère les données (sans doublons grâce à l'instruction DISTINCT) de la table temporary dans la table country. Elle sélectionne les colonnes country, region_code, et is_ldc de la table temporary, puis les insère dans la table country.

Résultat de la requête SQL `SELECT * FROM temporary ;`

country	year	civil_liberties	political_rights	status	region_code	region_name	is_ldc
Afghanistan	1995	7	7	NF	142	Asia	1
Afghanistan	1996	7	7	NF	142	Asia	1
Afghanistan	1997	7	7	NF	142	Asia	1
Afghanistan	1998	7	7	NF	142	Asia	1
Afghanistan	1999	7	7	NF	142	Asia	1
Afghanistan	2000	7	7	NF	142	Asia	1
Afghanistan	2001	7	7	NF	142	Asia	1
Afghanistan	2002	6	6	NF	142	Asia	1
Afghanistan	2003	6	6	NF	142	Asia	1
Afghanistan	2004	6	5	NF	142	Asia	1
Afghanistan	2005	5	5	PF	142	Asia	1
Afghanistan	2006	5	5	PF	142	Asia	1
Afghanistan	2007	5	5	PF	142-- Suite --		

Résultat de la requête SQL `SELECT * FROM country ;`

id_country	name	region_code	is_ldc
1	Monaco	150	0
2	Croatia	150	0
3	Nauru	9	0
4	Zambia	2	1
5	Montenegro	150	0
6	Indonesia	142	0
7	Yemen	142	1
8	Romania	150	0
9	Iraq	142	0
10	Philippines	142	0
11	Paraguay	19	0
12	Peru	19	0
13	Chad	2	1
14	Dominican Republic	19	0
15	Australia	9	0
16	Bahamas	19	0
17	Andorra	150	0
18	Georgia	142	0
19	Kyrgyzstan	142	0
20	Austria	150	0
21	Benin	2	1
22	Djibouti	2	1
23	Libya	2	0
24	Sri Lanka	142	0
25	Oman	142	0
26	Bahrain	142	0
27	Kuwait	142	0
28	Angola	2	1
29	Pakistan	142	0
30	Mozambique	2	1
31	Cambodia	142	1
32	Hungary	150	0
33	Guatemala	19	0
34	Cuba	19	0
35	Tajikistan	142	0
36	Nicaragua	19	0
37	C├âãÆ├é┬├te d├â┬ó├óÔÇÜ┬¼├óÔÇ×┬óIvoire	2	0
38	Algeria	2	0

-- Suite --

Résultat de la requête SQL `SELECT * FROM region ;`

```
 region_code |   name
-------------+----------
           2 | Africa
         142 | Asia
           9 | Oceania
          19 | Americas
         150 | Europe
(5 lignes)
```

Résultat de la requête SQL `SELECT * FROM status ;`

```
 status
--------
 PF
 NF
 F
(3 lignes)
```